AKTE W5·1

Fingerabdrücke

KERN LÖSUNG

ARCHIVFUND

Die gefälschte Update-Seite hat euch gezeigt: Ein Fingerabdruck verrät jede Veränderung einer Datei – wenn er etwas taugt. Die Archivarin legt euch ein selbst gebautes Prüfverfahren vor, den **KRYPTOS-Hash**, und fordert euch heraus: „Prüft, welche Anforderungen er erfüllt. Und findet heraus, wie Mallory ihn austricksen würde."

	offizielle Seite `archiv.chronoskop.example`	Datei aus der E-Mail
Datei	`chronoskop-update-v7.zip`	`chronoskop-update-v7.zip`
SHA-256	`03a20cc0cc22e8e2968e321a68205a4afc7aa9ba5e2d80aecaf9aaba0134dae7`	`03d0d35577d04206db84932bae7138627f2f3146cbb29110110bebc88f2ad39e`
Verbindung	Schloss-Symbol, Zertifikat für `archiv.chronoskop.example`	Schloss-Symbol, Zertifikat für `chronoskop-archiv.example`

Zur Erinnerung – der Archivfund: Nur die ersten zwei Zeichen stimmen überein, der Domainname ist ein anderer, und beide Seiten zeigen ein Schloss.

ARCHIVWISSEN 1 · HASHFUNKTIONEN UND IHRE ANFORDERUNGEN

Eine **Hashfunktion** ordnet jeder Nachricht beliebiger Länge einen **Hashwert fester Länge** zu (Fingerabdruck, Prüfwert). Kryptografische Hashfunktionen sollen:

Anforderung	Bedeutung
feste Länge	Hashwert immer gleich lang (SHA-256: 256 Bit = 64 Hexadezimalziffern)
effizient berechenbar	auch für große Dateien schnell
Einwegeigenschaft	zu einem Hashwert lässt sich praktisch keine passende Nachricht finden
Kollisionsresistenz	zwei verschiedene Nachrichten mit gleichem Hashwert lassen sich praktisch nicht finden
Lawineneffekt	eine winzige Änderung (ein Bit) ändert im Mittel etwa die Hälfte der Bits des Hashwerts

Beispiel SHA-256: „heute" → `b748a919 … 66b4bc7e`, „heutf" → `1751ffd9 … 79bd0dc3` – **134 der 256 Bits** sind verschieden.

Heute verwendet man **SHA-256** (SHA-2-Familie) oder **SHA-3**. **MD5 und SHA-1 gelten als gebrochen** – für beide sind praktische Kollisionen bekannt (MD5 ab 2004, SHA-1 2017).

ACHTUNG FALLE P9 · Hash ≠ Verschlüsselung

Beim Hashen gibt es **keinen Schlüssel**, und weil unendlich viele Nachrichten auf endlich viele Hashwerte abgebildet werden, kann man die Nachricht **nicht zurückgewinnen** – auch nicht die Besitzerin. Ein Hashwert dient zum **Prüfen** (Integrität), nicht zum Geheimhalten.

© Frank Juchim · Kryptos · für den Unterricht

ARCHIVWISSEN 2 · DER KRYPTOS-HASH (VON HAND)

Text normalisieren (nur Kleinbuchstaben a–z; ä → ae, ö → oe, ü → ue, ß → ss; Leerzeichen, Ziffern und Satzzeichen entfallen). **Nummerierung: A = 1 … Z = 26.** Start **h = 0**. Für jeden Buchstaben mit Wert w von links:

$$h \leftarrow (3 \cdot h + w) \bmod 29$$

Der letzte Zustand ist der Hashwert KH(text) ∈ {0, 1, …, 28}. Rechenvorlage am Beispiel „nebel“:

Buchstabe	Wert w	3 · h + w	h
n	14	0 + 14 = 14	14
e	5	42 + 5 = 47	18
b	2	54 + 2 = 56	27
e	5	81 + 5 = 86	28
l	12	84 + 12 = 96	**9**

Rechentipp: 3 · h + w ist höchstens 3 · 28 + 26 = 110 – ziehe also 29, 58 oder 87 ab. Nach **jeder** Runde reduzieren (P2).

Beispiel Runde 4: h = 27, w = 5 (e): 3 · 27 + 5 = 86; 86 − 58 = **28**.

Leerer Text: KH = 0. Zahlen immer als Wort schreiben – Ziffern entfallen beim Normalisieren (P4).

KH(„nebel“) = **9**

```
kryptosHash(text: Zeichenkette): Ganzzahl
  // text ist bereits normalisiert (nur Kleinbuchstaben a–z)
  h ← 0
  für i von 0 bis Länge(text) − 1
      w ← ASCII-Wert(Zeichen an Position i von text) − 96 // a = 1 … z = 26
      h ← (3 · h + w) mod 29
  gib h zurück
```

ARCHIVWISSEN 3 · KOLLISIONEN SIND UNVERMEIDLICH – UND MANCHMAL LEICHT

- **Schubfachprinzip:** Verteilt man mehr Gegenstände als Schubfächer, liegen in einem Fach mindestens zwei. Der KRYPTOS-Hash hat 29 „Fächer“: Unter 30 verschiedenen Nachrichten haben **sicher** zwei denselben Hash. Jede Hashfunktion hat Kollisionen – gute Hashfunktionen machen sie nur **unauffindbar**.
- **Gezielte Kollision beim KRYPTOS-Hash:** Der vorletzte Buchstabe geht mit Faktor 3, der letzte mit Faktor 1 in den Hash ein. Erhöht man den **vorletzten um 1** und verringert den **letzten um 3**, ändert sich nichts: „nebel“ → „neb**fi**“ (e → f, l → i).

Zustände	1	2	3	4	5
nebel	14	18	27	28	**9**
nebfi	14	18	27	0	**9**

© Frank Juchim · Kryptos · für den Unterricht

ARCHIVWISSEN 4 · PRÜFSUMMEN UND PASSWÖRTER

- **Summen-Prüfsumme** $S(t) = (\Sigma\, w_i) \bmod 29$: erkennt geänderte Buchstaben, aber **keine Vertauschungen** (permutationsinvariant).
- **Gewichtete Prüfsumme** $G(t) = (1 \cdot w_1 + 2 \cdot w_2 + 3 \cdot w_3 + \ldots) \bmod 29$: Vertauschungen ändern den Wert.
- **Passwort-Hashing:** Bei der Registrierung erzeugt der Dienst ein zufälliges **Salz** und speichert Salz und Hash(Salz + Passwort), **nie das Passwort**. Beim Login hängt er das Salz an das eingegebene Passwort, hasht und vergleicht. Gleiche Passwörter ergeben so verschiedene gespeicherte Werte, vorab berechnete Tabellen nutzen Angreifern nichts. In der Praxis: absichtlich langsame Verfahren wie **Argon2** oder **bcrypt**.

1 AUFTRAG ★ Zwei Wörter, ein Fingerabdruck? EINZELARBEIT · 8 MIN

Berechne KH(„anker“) und KH(„tor“) mit der Rechenvorlage. Was fällt auf? Welche Anforderung ist damit verletzt?

LÖSUNG P2

Buchstabe	Wert w	3 · h + w	h
a	1	0 + 1 = 1	1
n	14	3 + 14 = 17	17
k	11	51 + 11 = 62	4
e	5	12 + 5 = 17	17
r	18	51 + 18 = 69	**11**

Buchstabe	Wert w	3 · h + w	h
t	20	0 + 20 = 20	20
o	15	60 + 15 = 75	17
r	18	51 + 18 = 69	**11**

Beide Wörter haben den Hashwert **11**: gleicher Hashwert für verschiedene Nachrichten = **Kollision** → die **Kollisionsresistenz** ist verletzt.

Typischer Fehler: 62 mod 29 = 33 (nur einmal 29 abgezogen, P2).

2 AUFTRAG ★★ Ein Buchstabe Unterschied PARTNERARBEIT · 8 MIN

a) Berechne KH(„kiste“) und KH(„kistf“).

b) **Untersuche**, ob der KRYPTOS-Hash den Lawineneffekt erfüllt, und vergleiche mit SHA-256 (Archivwissen 1).

c) **Begründe**, warum ein fehlender Lawineneffekt einem Angreifer hilft.

© Frank Juchim · Kryptos · für den Unterricht

LÖSUNG

Buchstabe	Wert w	3 · h + w	h
k	11	0 + 11 = 11	11
i	9	33 + 9 = 42	13
s	19	39 + 19 = 58	0
t	20	0 + 20 = 20	20
e	5	60 + 5 = 65	**7**

Buchstabe	Wert w	3 · h + w	h
k	11	0 + 11 = 11	11
i	9	33 + 9 = 42	13
s	19	39 + 19 = 58	0
t	20	0 + 20 = 20	20
f	6	60 + 6 = 66	**8**

a) KH(„kiste“) = **7**, KH(„kistf“) = **8**.

b) Eine Änderung des letzten Buchstabens um 1 ändert den Hash nur um 1 → **kein Lawineneffekt**. SHA-256: ein Buchstabe anders („heute“/„heutf“) → 134 von 256 Bits verschieden → Lawineneffekt erfüllt.

c) Ohne Lawineneffekt sieht man, wie sich Änderungen auswirken, und kann sie gezielt ausgleichen – also Kollisionen konstruieren (Auftrag 4).

3 AUFTRAG ★★ Schubfächer EINZELARBEIT · 6 MIN

a) **Begründe**, dass es beim KRYPTOS-Hash Kollisionen geben muss, ohne eine einzige zu berechnen.

b) Wie viele verschiedene Nachrichten braucht man mindestens, um bei SHA-256 **mit Sicherheit** eine Kollision zu haben?

c) Warum ist SHA-256 trotzdem kollisionsresistent?

LÖSUNG

a) Es gibt nur 29 mögliche Hashwerte, aber unendlich viele Nachrichten – nach dem **Schubfachprinzip** müssen mehrere Nachrichten denselben Hashwert haben.

b) $2^{256} + 1$ Nachrichten.

c) Kollisionen **existieren**, aber es ist kein Verfahren bekannt, eine zu **finden**; Ausprobieren bräuchte etwa 2^{128} Versuche (Geburtstagseffekt) – praktisch unmöglich.

4 AUFTRAG ★★★ Mallorys Kollisionsbaukasten PARTNERARBEIT · 12 MIN

a) **Zeige** allgemein: Ist h der Zustand vor dem vorletzten Buchstaben (Wert v) und dem letzten Buchstaben (Wert l), dann ist der Hashwert $(9 \cdot h + 3 \cdot v + l) \bmod 29$. Folgere daraus die Regel „vorletzter +1, letzter −3“.

b) KH(„post“) = 6. **Bestimme** mit der Regel eine Kollision und prüfe sie mit einer Rechenvorlage.

c) **Bestimme** eine Nachricht, die mit „ab“ beginnt und den KRYPTOS-Hash 20 hat. Welche Anforderung ist damit verletzt?

© Frank Juchim · Kryptos · für den Unterricht

LÖSUNG P2

a) Vorletzte Runde: $h_1 = 3h + v$, letzte Runde: $h_2 = 3h_1 + l = 9h + 3v + l$ (jeweils mod 29). Mit $v + 1$ und $l - 3$: $9h + 3(v + 1) + (l - 3) = 9h + 3v + l$ → **gleicher Hash** (solange $v + 1 \le 26$ und $l - 3 \ge 1$).

b) „post“: s → t, t → q ergibt **„potq“**:

Buchstabe	Wert w	3 · h + w	h
p	16	0 + 16 = 16	16
o	15	48 + 15 = 63	5
t	20	15 + 20 = 35	6
q	17	18 + 17 = 35	**6**

c) Gesucht w mit $(3 \cdot 5 + w) \bmod 29 = 20$ → w = 5 = e:

Buchstabe	Wert w	3 · h + w	h
a	1	0 + 1 = 1	1
b	2	3 + 2 = 5	5
e	5	15 + 5 = 20	**20**

c) KH(„ab“) = 5 (Zustände 1, 5); w = 20 − 15 = 5 → **„abe“** hat KH = 20. Die **Einwegeigenschaft** ist verletzt: Zu einem Hashwert lässt sich direkt eine Nachricht berechnen.

5 AUFTRAG ★★ Rabe oder Bear? PARTNERARBEIT · 8 MIN

Ein Lager vergibt Codewörter. Statt „rabe“ kommt beim Empfänger „bear“ an (Buchstaben vertauscht).

a) Berechne S und G für beide Wörter.

b) **Untersuche**, welche Prüfsumme den Fehler bemerkt.

c) Berechne auch den KRYPTOS-Hash beider Wörter und **erläutere**, warum er die Vertauschung bemerkt.

LÖSUNG

Wort	Werte w_i	S(t)	G(t)	KH(t)
rabe	18, 1, 2, 5	26	18 + 2 + 6 + 20 = 46 → **17**	18 → 26 → 22 → **13**
bear	2, 5, 1, 18	26	2 + 10 + 3 + 72 = 87 → **0**	2 → 11 → 5 → **4**

b) Nur **G** bemerkt die Vertauschung; S ist **permutationsinvariant** (die Summe hängt nicht von der Reihenfolge ab).

c) KH(„rabe“): 18 → 55 → 26 → 80 → 22 → 71 → **13**; KH(„bear“): 2 → 11 → 34 → 5 → 33 → **4**. Jeder Buchstabe geht mit einem **positionsabhängigen Faktor 3^j** ein, daher ändern Vertauschungen in der Regel den Hash.

6 AUFTRAG ★★ Salz in der Datenbank EINZELARBEIT · 10 MIN

Ein Dienst nutzt (nur zum Üben) den KRYPTOS-Hash für Passwörter. Gespeichert ist:

Konto	Salz	gespeicherter Wert KH(Salz + Passwort)
Ole	qx	13
Kim	kz	11

a) Kim meldet sich mit dem Passwort „sonne“ an. **Beschreibe** die Prüfung des Servers und führe sie durch.

b) Ole hat dasselbe Passwort. Woran erkennt ein Angreifer mit Zugriff auf die Datenbank das **nicht**?

© Frank Juchim · Kryptos · für den Unterricht

c) Ein zweiter Dienst speichert **ohne Salz**: KH(„sonne“) = 0. Ein Angreifer probiert das Passwort „geheim“. **Untersuche**, ob er hineinkommt, und **erläutere**, was das über die Länge echter Passwort-Hashes aussagt.

LÖSUNG P9

a) KH(„kz“ + „sonne“):

Buchstabe	Wert w	3 · h + w	h
k	11	0 + 11 = 11	11
z	26	33 + 26 = 59	1
s	19	3 + 19 = 22	22
o	15	66 + 15 = 81	23
n	14	69 + 14 = 83	25
n	14	75 + 14 = 89	2
e	5	6 + 5 = 11	**11**

c) KH(„geheim“):

Buchstabe	Wert w	3 · h + w	h
g	7	0 + 7 = 7	7
e	5	21 + 5 = 26	26
h	8	78 + 8 = 86	28
e	5	84 + 5 = 89	2
i	9	6 + 9 = 15	15
m	13	45 + 13 = 58	**0**

a) Der Server liest Kims Salz „kz“, hängt das eingegebene Passwort an („kzsonne“), berechnet KH = **11** und vergleicht mit dem gespeicherten Wert 11 → Anmeldung erfolgreich.

b) Durch das unterschiedliche Salz sind die gespeicherten Werte verschieden (13 bzw. 11).

c) KH(„geheim“) = **0** = KH(„sonne“) → der Angreifer kommt mit einem **falschen** Passwort hinein (Kollision). Echte Passwort-Hashes müssen so lang sein (z. B. 256 Bit), dass solche Zufallstreffer praktisch ausgeschlossen sind.

7 AUFTRAG ★ Das verschlüsselte Tagebuch? EINZELARBEIT · 4 MIN

„Ich speichere meine Tagebucheinträge als SHA-256-Hash, dann sind sie verschlüsselt.“ **Nimm Stellung** mit zwei fachlichen Argumenten.

LÖSUNG P9

Falsch: (1) Ein Hash hat **keinen Schlüssel** – die Einträge können auch von der Besitzerin nicht wiederhergestellt werden; (2) viele Texte haben denselben Hashwert, eine eindeutige Rückgewinnung ist unmöglich. Geeignet wäre eine Verschlüsselung (z. B. AES mit einem aus dem Passwort abgeleiteten Schlüssel). Ein Hash eignet sich nur zum **Prüfen** (Integrität).

© Frank Juchim · Kryptos · für den Unterricht

ABI-CHECK · ca. 20 min 10 BE

a) Ein Mitschüler schlägt vor, vertrauliche Nachrichten zu schützen, indem statt der Nachricht nur ihr SHA-256-Hashwert gesendet wird. **Beurteilen Sie** diesen Vorschlag. **[3 BE]**

ERWARTUNG a) P9 P10

Der Empfänger kann aus dem Hashwert die Nachricht nicht zurückgewinnen, da es keinen Schlüssel gibt und die Funktion nicht umkehrbar ist (1); wegen der festen Länge haben unendlich viele Nachrichten denselben Hashwert, die Nachricht ist also nicht eindeutig bestimmt (1); Urteil: ungeeignet – Hashfunktionen dienen der Integritätsprüfung, für Vertraulichkeit braucht man Verschlüsselung (1).

b) **Bestimmen Sie** den KRYPTOS-Hash der Nachricht „hafen“. **[2 BE]**

ERWARTUNG b) P2

8 → 24 + 1 = 25 → 75 + 6 = 81 → 23 → 69 + 5 = 74 → 16 → 48 + 14 = 62 → **4** (Rechenweg 1, Ergebnis 1).

c) **Untersuchen Sie**, ob der KRYPTOS-Hash den Lawineneffekt erfüllt. **[2 BE]**

ERWARTUNG c)

Beispiel: Eine Änderung des letzten Buchstabens um 1 ändert den Hashwert nur um 1, z. B. KH(„kiste“) = 7, KH(„kistf“) = 8 (1); der Lawineneffekt ist nicht erfüllt (1).

d) **Beschreiben Sie**, wie ein Online-Dienst Passwörter mithilfe einer Hashfunktion und eines Salzes prüfen kann, ohne sie im Klartext zu speichern. **[3 BE]**

ERWARTUNG d)

Registrierung: zufälliges Salz erzeugen, Hash(Salz + Passwort) und Salz speichern (1); Login: Salz laden, an das eingegebene Passwort anhängen, hashen (1); mit dem gespeicherten Wert vergleichen, bei Gleichheit Zugang (1).

IM LAB ÜBEN Level 5.1

Level 5.1 „Fingerabdrücke“ im Kryptolabor mit dem **Hash-Labor** (Lawinen-Vergleich, Kollisionssuche). Trainingsplatz: „KRYPTOS-Hash“ (5 Aufgaben), „Kollisionsbau“ (3 Aufgaben), „Prüfsummen“ (2 Aufgaben).

© Frank Juchim · Kryptos · für den Unterricht